

POO - testul 2

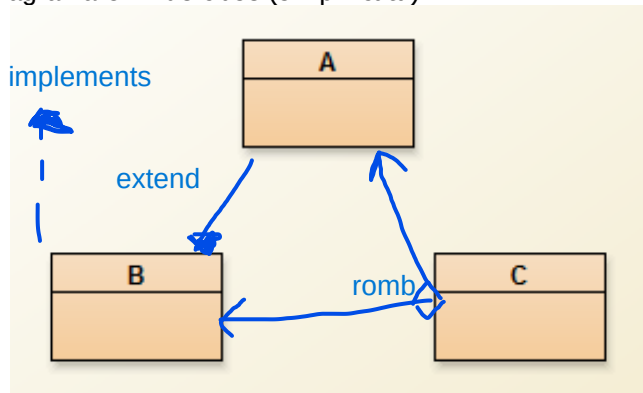
1. (3.9p = 10 grile x 0.3p + 0.9p) Completați în grila alăturată, în ordine, cele 10 valori afișate pe rânduri separate în urma execuției următorului program JAVA (3p). Desenați pe diagrama UML simplificată de mai jos toate relațiile care există între cele trei clase (0.9p).

```
public class A extends B{
    int valoare;
    static {
        info+=getInfo();
    }
    public A() {
        nume+="Craciun ";
        valoare+=25;
    }
    public String toString() {
        return "Cadouri ";
    }
    public static String getInfo() {
        return "6 ";
    }
}
```

```
public class B {
    String nume = "Mos ";
    int valoare = 10;
    public static String info = getInfo();
    public B() {
        nume+="Nicolae ";
        valoare--;
    }
    public String toString() {
        return "Surprize ";
    }

    public static String getInfo() {
        return "Decembrie ";
    }
}
```

Diagrama UML de clase (simplificată):



```
public class C {
    private A a;
    private B b;
    public C() {
        a = new A();
        b = a;
    }
    public static void main(String[] args) {
        C obj = new C();
        System.out.println(obj.a.valoare);
        System.out.println(obj.b.valoare);
        System.out.println(obj.a.nume);
        System.out.println(obj.b.nume);
        System.out.println(obj.a);      toString
        System.out.println(obj.b);
        System.out.println(obj.a instanceof B);
        System.out.println(obj.b instanceof A);
        System.out.println(obj.b.getClass().getName());
        System.out.println(A.info);
    }
}
```

Grila de răspunsuri:

01	25
02	9
03	Mos Nicolae Craciun
04	Mos Nicolae Craciun
05	Cadouri
06	Cadouri
07	true
08	true
09	A
10	Decembrie 6

2. (5.1p = 17x0.3p) Completați în grila de mai jos conținutul care lipsește din chenarele numerotate de la 01 la 17 din programul JAVA scris mai jos astfel încât să respectați paradigma de POO, iar programul să afișeze în urma execuției următoarele rezultate:

Nu se pot adauga vagoane de marfa la un tren de calatori si nici vagoane de calatori la un tren de marfa!
Tren cu urmatoarele vagoane (in ordine invers lexicografica a identificatorilor lor):
Vagon: ID=87453 Tip:VagonCalatori Nr.locuri:110
Vagon: ID=23143 Tip:VagonCalatori Nr.locuri:120
Tren cu urmatoarele vagoane (in ordine invers lexicografica a identificatorilor lor):
Vagon: ID=89942 Tip:VagonMarfa MasaMaxima:4549.0 Volum:48.0
Vagon: ID=24896 Tip:VagonMarfa MasaMaxima:5786.0 Volum:54.0
Vagon: ID=14234 Tip:VagonMarfa MasaMaxima:5667.0 Volum:53.0

```
public abstract class Vagon [01] Comparable{
    private String identificador;
    public Vagon(String identificador) {
        this.identificador=identificador;
    }
    public String getIdentificador() {
        return identificador;
    }
    public String toString() {
        return "Vagon: "+"ID="+identificador+" Tip:"+[02];
    }
    public int [03](Object obj) {
        return [04];
    }
}

public class VagonCalatori [05] {
    private int nrLocuri;
    public VagonCalatori(String identificador, int nrLocuri) {
        [06];
        this.nrLocuri=nrLocuri;
    }
    public String toString() {
        return [07] + " Nr.locuri:"+nrLocuri;
    }
}

public class VagonMarfa [05] {
    private float masaMaxima;
    private float volum;
    public VagonMarfa(String identificador,
        float masaMaxima, float volum) {
        [06];
        this.masaMaxima=masaMaxima; this.volum=volum;
    }
    public String toString() {
        return [07] + " MasaMaxima:"+masaMaxima+ " Volum:"+volum;
    }
}

[08] java.util.Arrays;
public class Tren{
    private Vagon[] vagoane;
    private int nrVagoaneInGarnitura, nrMaximVagoane;
    public Tren(int nrMaximVagoane) {
        this.nrMaximVagoane=nrMaximVagoane;
        vagoane = new Vagon[nrMaximVagoane];
    }
    public void addVagon(Vagon v) [09] {
        if (nrVagoaneInGarnitura<nrMaximVagoane) {
            if (nrVagoaneInGarnitura>0 && v.[10]!=vagoane[0].[10])
                [11]("Nu se pot adauga vagoane de marfa la un tren
                de calatori si nici vagoane de calatori la un
                tren de marfa!");
            vagoane[nrVagoaneInGarnitura++]=v;
        }
    }
    private void sorteazaVagoane() {
        [12].sort(vagoane,0,nrVagoaneInGarnitura);
        /* sorteaza tabloul de la indexul 0 (inclusiv)
        pana la indexul nrVagoaneInGarnitura (exclusiv) */
    }
    public String toString() {
        String s = "Tren cu urmatoarele vagoane (in ordine
        invers lexicografica a identificatorilor lor):\n";
        [13];
        for (Vagon v:vagoane)
            if (v!=null) s+=v+"\n";
        return s; }
}
```

```
public class MainClass {
    public static void main (String [] args) [09] {
        Tren tCalatori = new Tren(15);
        Tren tMarfa = new Tren(35);

        [14]{
            tCalatori.addVagon(new VagonCalatori("23143",120));
            tCalatori.addVagon(new VagonCalatori("87453",110));
            tCalatori.addVagon(
                new VagonMarfa("45896", 5468.5[15], 45.8[15]));
            tCalatori.addVagon(new VagonCalatori("34587",95));
        }[16]{
            System.out.println(e.[17]);
        }

        tMarfa.addVagon(new VagonMarfa("24896",5786,54));
        tMarfa.addVagon(new VagonMarfa("89942",4549,48));
        tMarfa.addVagon(new VagonMarfa("14234",5667,53));

        System.out.println(tCalatori);
        System.out.println(tMarfa);
    }
}
```

Grila de răspunsuri:

01	implements
02	getClass().getName()
03	CompareTo
04	((Vagon)obj).identificador.compare To(identificador)
05	extends Vagon
06	super(identificador)
07	super.toString()
08	import
09	throws Exception
10	getClass()
11	throw new Exception
12	Arrays.sort
13	sorteazaVagoane()
14	try
15	f
16	catch
17	getMessage()